Task 2 – Dynamic Prompt Builder Service

Objective

The objective of this task was to understand Dynamic Prompt Engineering and Prompt Structuring by creating a service that converts structured metadata into optimized LLM-ready prompts.

Problem Statement

Raw text prompts are often inconsistent and difficult to maintain. To solve this problem, a Dynamic Prompt Builder Service was developed that accepts structured JSON input and automatically generates a standardized prompt template suitable for AI models.

Technologies Used

- Python
- FastAPI
- HTML
- CSS
- JavaScript
- Fetch API

Implementation

Backend Development

A FastAPI backend was created with a POST endpoint named /build-prompt.

The endpoint accepts a JSON payload containing:

```
{
  "project": "Inventory System",
  "skills": ["Python", "FastAPI"]
}
```

The backend processes the data and dynamically constructs a prompt using string templates.

Example generated prompt:

Generate ATS friendly bullet points.

Project:

Inventory System

Skills:

Python, FastAPI

The generated prompt is returned as a JSON response.

Frontend Development

A modern user interface was developed using HTML, CSS, and JavaScript.

Features implemented:

- Responsive design
- Modern glassmorphism UI
- User-friendly form inputs
- API integration using Fetch API
- Dynamic display of generated prompts
- Loading animation during API requests

The frontend collects project details and skills from the user and sends them to the FastAPI backend.

API Workflow

1. User enters project information.
2. User enters required skills.
3. Frontend sends a POST request to the FastAPI endpoint.
4. Backend generates a structured prompt.
5. Generated prompt is returned as JSON.
6. Frontend displays the generated prompt to the user.

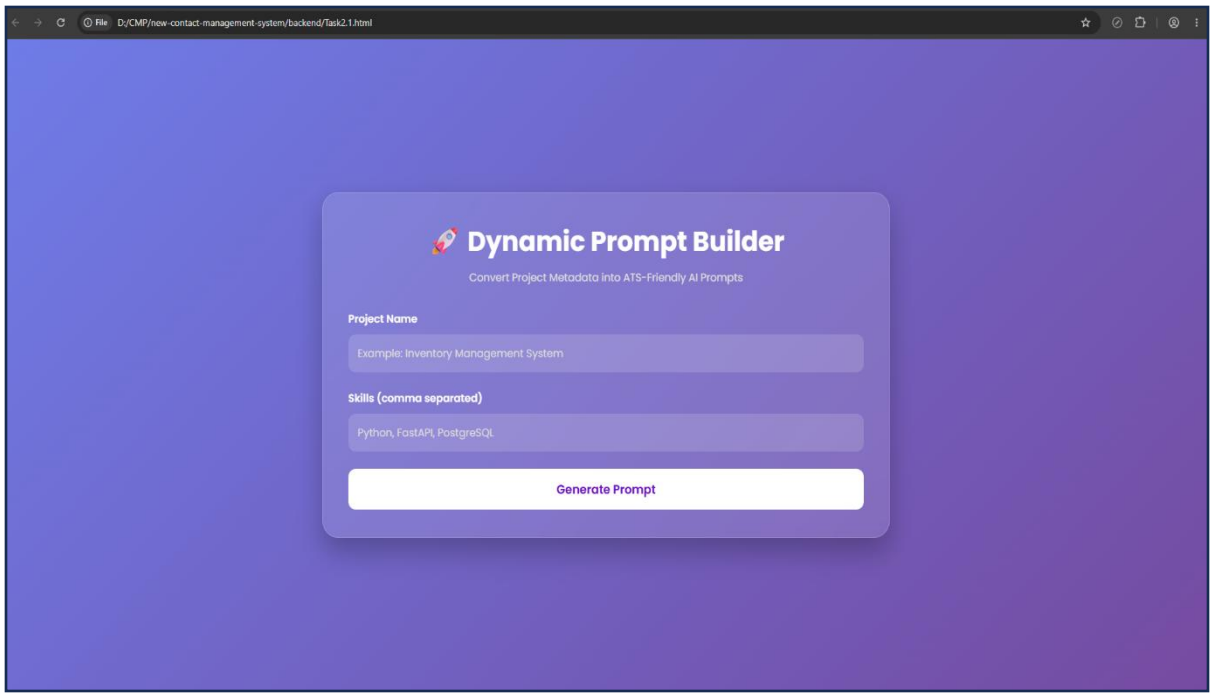
Learning Outcomes

Through this task, the following concepts were learned:

- Dynamic prompt generation
- Prompt engineering fundamentals
- FastAPI endpoint creation

- JSON request and response handling
- Frontend and backend communication
- Fetch API integration
- Template-based string construction
- Building AI-ready context from structured metadata

Screenshot 1: Frontend User Interface



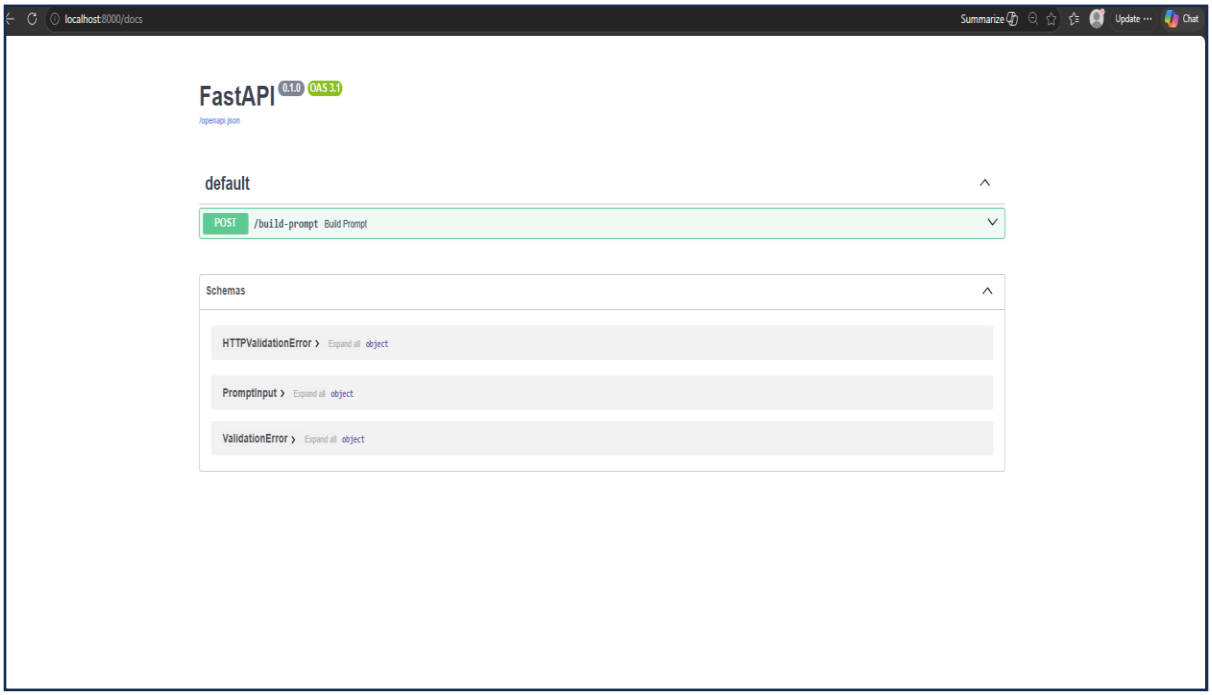
Screenshot 2 : FastAPI Source Code (Backend)

```python
from fastapi import FastAPI
from pydantic import BaseModel
from typing import List
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

class PromptInput(BaseModel):
    project: str
    skills: List[str]

@app.post("/build-prompt")
def build_prompt(data: PromptInput):

    prompt = f"""Generate ATS friendly bullet points.

Project:
{data.project}

Skills:
{", ".join(data.skills)}
"""

    return {
        "prompt": prompt
    }
```

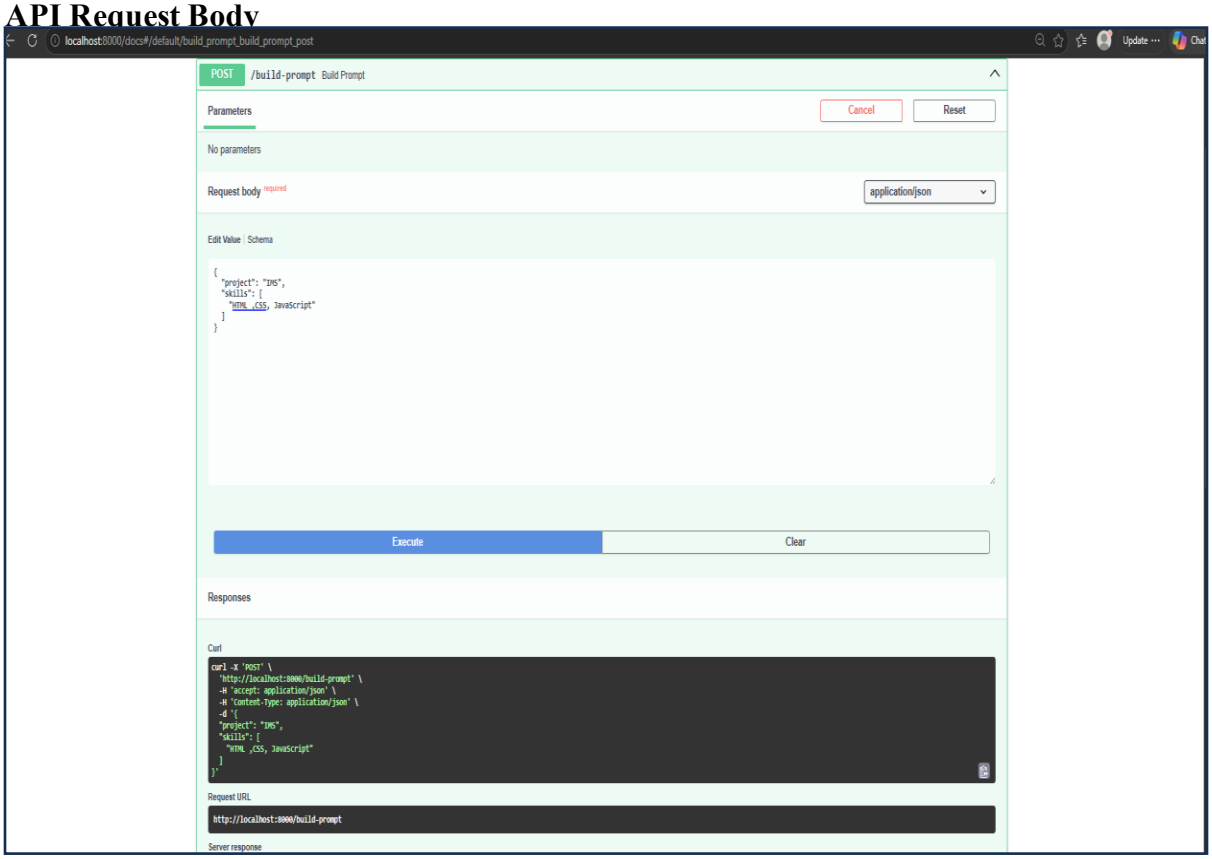
(Frontend Code)

```html
<html lang="en">
<body>
<script>
async function generatePrompt(){

        const response = await fetch(
            "http://127.0.0.1:8000/build-prompt",
            {
                method:"POST",
                headers:{
                    "Content-Type":"application/json"
                },
                body:JSON.stringify({
                    project:project,
                    skills:skills
                })
            }
        );

        const data = await response.json();

        document.getElementById("output").textContent =
            data.prompt;

        document.getElementById("outputCard")
            .style.display="block";

    }
    catch(error){

        document.getElementById("output").textContent =
            "Error connecting to API";

        document.getElementById("outputCard")
            .style.display="block";
    }

    document.getElementById("loader").style.display="none";
}
```

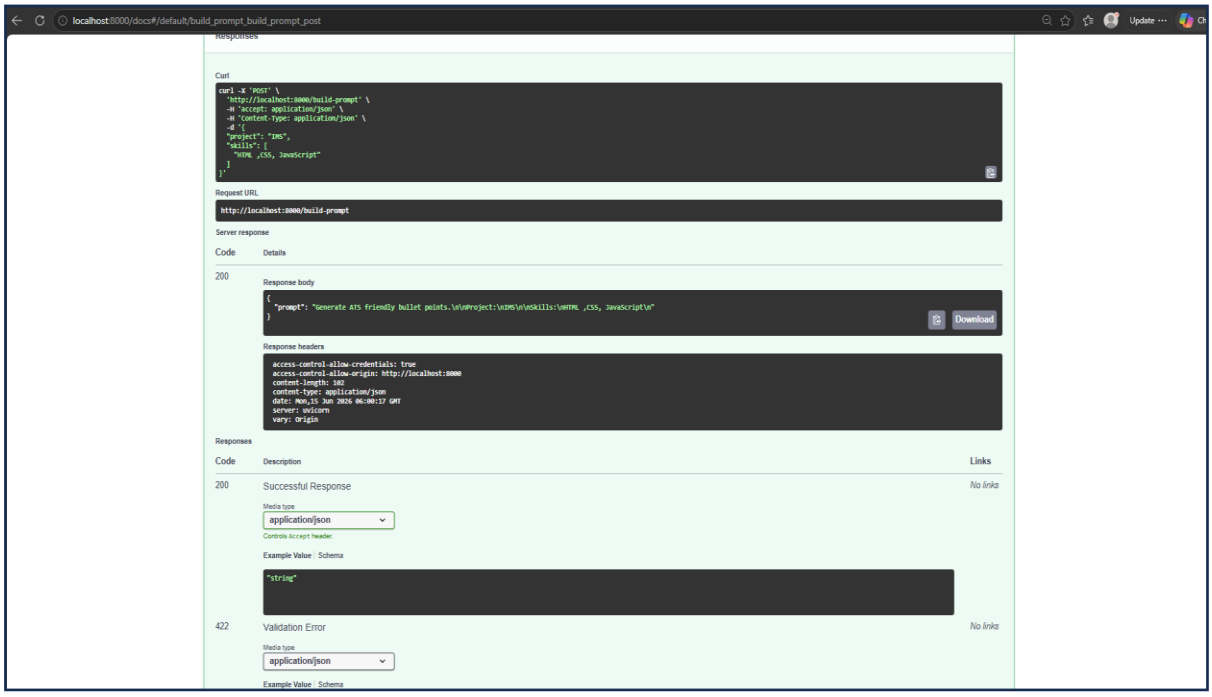
Screenshot 3 : API Endpoint Testing using FastAPI Swagger UI



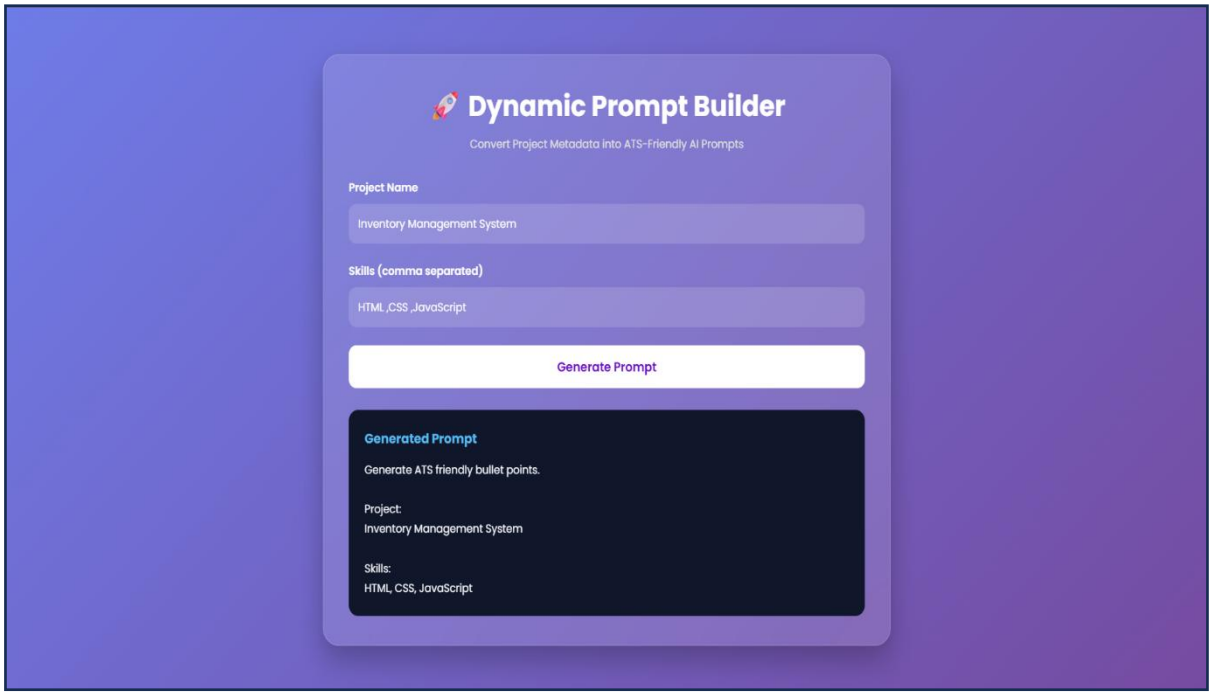
API Request Body



API Responses



Generated Output Displayed



Challenges Faced

- Configuring communication between frontend and backend.
- Handling Cross-Origin Resource Sharing (CORS) issues.
- Formatting dynamic data into a consistent prompt structure.
- Displaying API responses dynamically on the frontend.

Conclusion

This task provided hands-on experience in prompt engineering, API development, and frontend-backend integration. The developed solution effectively transforms structured input data into reusable and consistent LLM-ready prompts, improving prompt quality and maintainability for AI-based applications.